Software Design Analysis Report

Proposed design based on Software Requirements Specification (SRS v1.0, 13 September 2026)

A. Project Details

Field	Details
Project title	AI-Powered Academic Advisor
Team members	Divyanshi Gupta; Shreya Singh; Tanmay Shekhar; Harshit Singh Bharti
Project objective	Provide personalized academic guidance using student profiles, performance records and assessments. Identify weak areas, recommend subjects/resources, generate study plans and monitor progress with interpretable recommendations.
Technology stack (proposed; not finalized)	Responsive web frontend; backend REST API; persistent student/academic database; AI/ML recommendation service; HTTPS and JSON interfaces. Specific frameworks, hosting and model have not been selected.

B. Module Identification and Modularization

Total number of major modules: 7. The boundaries group closely related requirements and separate data entry, intelligence, guidance and administration. Design is proposed from the SRS; modules have not yet been implemented.

ID	Module	Main responsibility	Inputs / outputs	Depends on
M1	Access & Profile	Authenticate users; manage learner profile (FR1-2).	Credentials/profile -> authorized identity/profile.	Auth store
M2	Academic Records	Collect marks, grades, attendance (FR3).	Academic entries -> validated records.	M1, DB
M3	Assessment	Record and evaluate student assessment (FR4).	Responses -> assessment summary.	M1, DB
M4	AI Analysis & Advice	Find strengths/gaps; score, rank and explain recommendations (FR5-7).	M2/M3 data + goals -> ranked advice.	M2, M3, M7
M5	Study Plan & Resources	Build personalized plans and suggest resources (FR8-9).	M4 advice + time -> plan/resources.	M4, M7
M6	Progress & Dashboard	Track completion and display insights (FR10-11).	Status updates -> progress/overview.	M2, M4, M5
M7	Admin & Knowledge	Maintain subjects, rules, skill maps and resources (FR12).	Admin changes -> curated catalog.	M1, DB

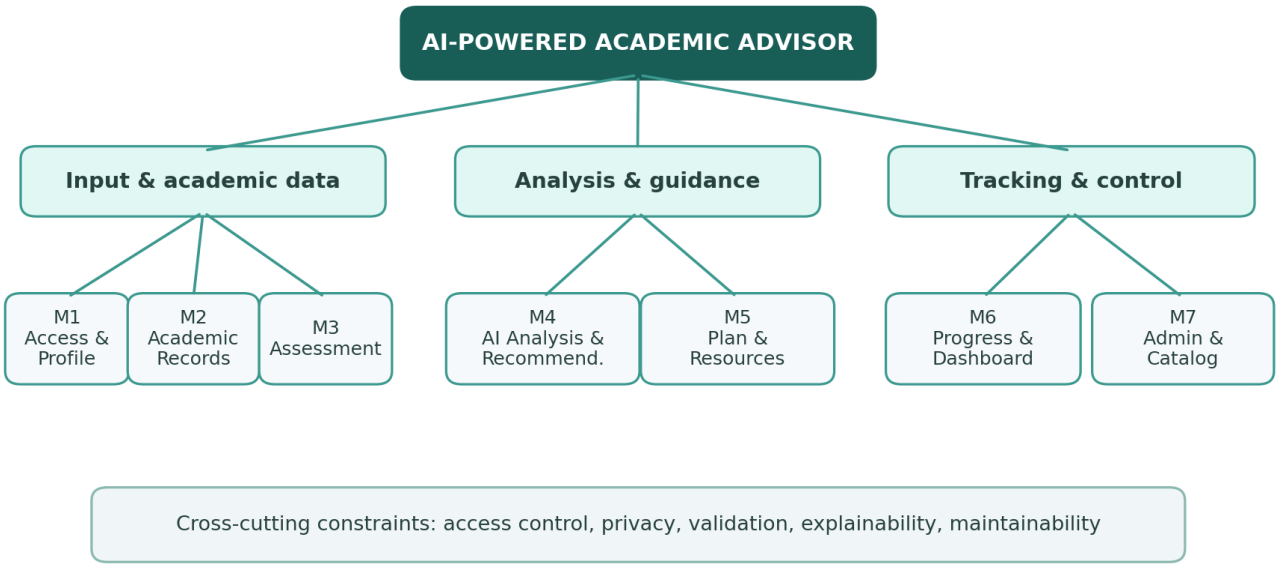


Figure 1. Proposed major-module decomposition (M1-M7).

C. Design Strategy

Selected strategy: Top-Down (proposed)

Justification: The SRS defines the overall advisory goal and a Level-0 context, then decomposes it into Level-1 processes and FR1-FR12. Top-down design fits this progression because system goals become services, inputs/outputs and smaller operations. It enables traceability from requirements to modules before the stack is fixed. Each service can then be implemented and tested separately while keeping a clear integration contract.

Evidence / design procedure: Start with the student-to-advice workflow (SRS §2.3), identify seven logical modules from the functions and traceability matrix (SRS §§3, 10), refine interfaces from core data entities (SRS §6.3), and assign validation and access controls from NFR1, NFR8 and NFR9. This is a proposed decomposition procedure, not evidence of completed implementation.

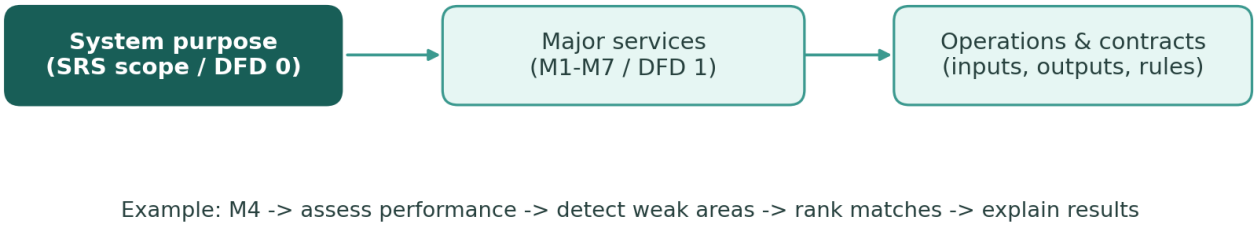


Figure 2. Top-down path from system requirements to operations.

D. Design Approach

Selected approach: Function-Oriented (proposed)

Justification: The original SRS organizes behavior as flows and named processes: collect academic data, perform analysis, create recommendations, generate plans and track progress. A function-oriented structure chart directly reflects these transformations and aligns with the DFDs. The student, subject, assessment and recommendation entities remain structured data inputs and outputs. This is a design choice for analysis; the eventual implementation may use classes and objects internally.

Evidence / structure chart: The high-level advice operation delegates to data retrieval, performance analysis, scoring/explanation, plan generation and dashboard presentation. Sub-functions use defined outputs rather than shared mutable global state.

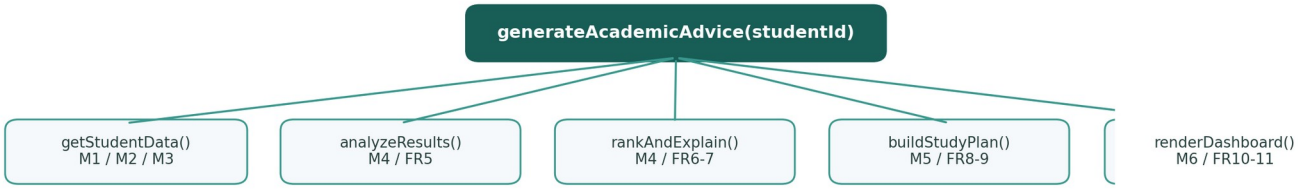


Figure 3. Proposed function-oriented structure chart; FRs map to the uploaded SRS.

E. Coupling Analysis

Module connections are proposed API/data contracts. Their coupling labels describe the intended design, not a measured property of implemented code. The goal is to exchange only necessary values and avoid hidden shared state.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M2 -> M4	Validated marks, grades, attendance, subjects.	Data coupling / low (target)	Pass a minimal, versioned academic-summary DTO.
M3 -> M4	Assessment scores and stated preferences.	Data coupling / low (target)	Use aggregated results and schema validation; avoid raw form internals.
M4 -> M5	Ranked recommendations, priorities, reasons.	Data coupling / low (target)	Return documented recommendation result; isolate scoring algorithm.
M5 -> M6	Plan identifier, activities, completion status.	Data coupling / low (target)	Use stable plan/progress APIs; avoid direct access to M5 internals.
M7 -> M4	Subject mappings, prerequisites, academic rules.	Potential stamp coupling / medium	Expose only needed catalog fields via a versioned query interface.

Coupling assessment: The intended design favors low data coupling. It explicitly avoids common coupling through shared global data and uses access controls when multiple modules read the academic store. Catalog requests (M7 -> M4) need special care because passing a whole knowledge-base object would create unnecessary stamp coupling.

F. Cohesion Analysis

Each module is organized around a primary business purpose. The ratings below are design expectations; they should be revisited after implementation and testing.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
M2 Academic Records	Enter, validate and retrieve academic records.	Functional / high	Keep reporting/AI decisions outside this module.
M4 AI Analysis	Analyze performance -> detect gaps -> rank and explain advice.	Sequential / high	Make scoring and explanation functions independently testable.
M5 Plan & Resources	Generate plan and pair it with learning material.	Communicational / medium-high	Split resource lookup from scheduling if complexity expands.
M6 Progress & Dashboard	Update completion and show progress information.	Communicational / medium-high	Separate progress-write service from dashboard read layer later.

G. Module Interaction and Design Quality

The sequence below describes the planned "Get Academic Advice" use case, traced to SRS FR3-FR11. The learner sends a request; authenticated backend operations collect records and assessment summaries, query curated subject rules, generate explainable advice and assemble study guidance for the dashboard. It is a proposed interaction, not a test-run trace.

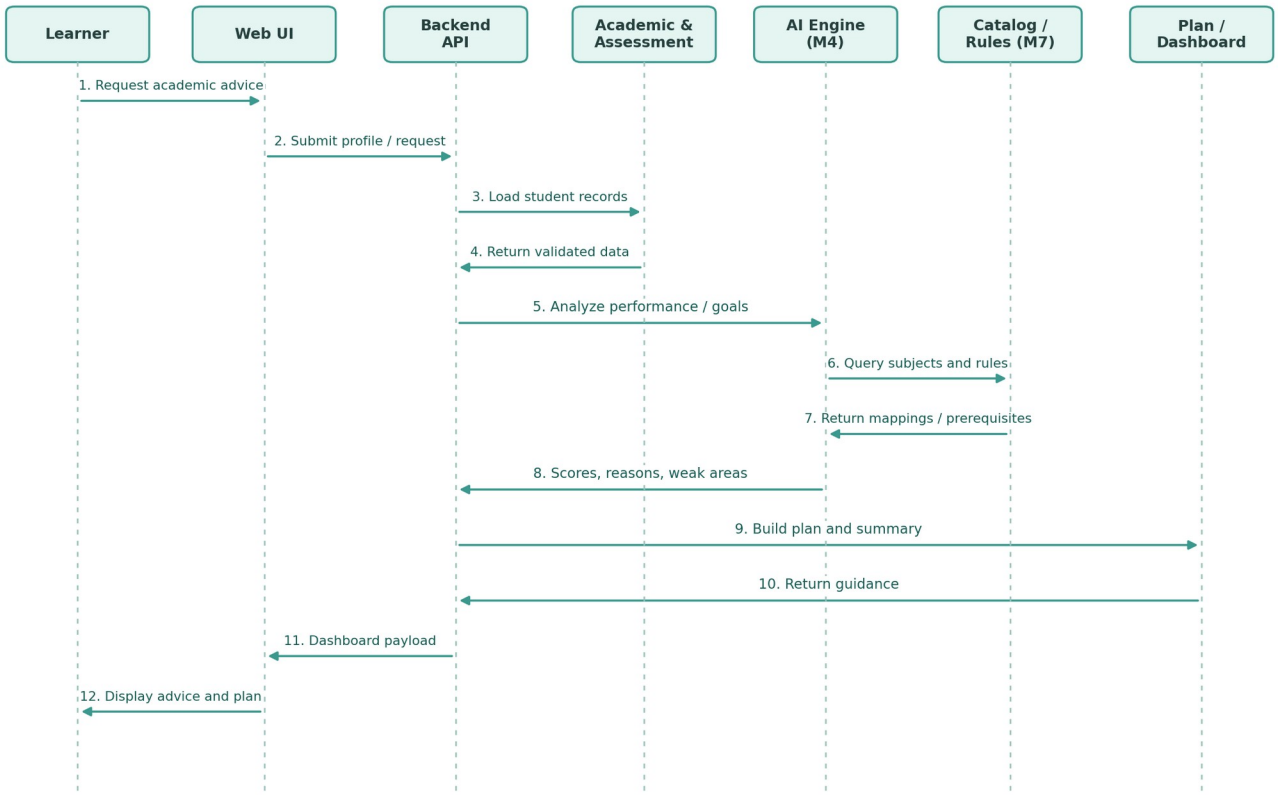


Figure 4. Proposed sequence diagram - learner requests a recommendation and study plan.

Design-quality observations

Quality aspect	Design decision and SRS alignment
Low coupling / high cohesion	Separate academic records, assessments, recommendation processing, planning and progress; communicate through validated, minimal API contracts (NFR6).
Security and privacy	Enforce role-based access to student data; require authenticated API calls; use HTTPS and least-privilege permissions (NFR1, NFR9; SRS §5.3).
Explainability / reliability	Return recommendation reasons and score factors; check incomplete profiles before final advice (NFR3, NFR8; SRS decision rule R5).
Change resilience	Keep subject/prerequisite rules in an administered catalog so curricula and resource links can be updated without rewriting recommendation workflows (FR12).

Design status and basis: Planning only. The proposed modules, strategy, function structure, coupling and cohesion assessments, and sequence flow are derived from AI-Powered Academic Advisor SRS v1.0 (13/09/2026, especially §§2-3, 5-10) and the instructor's Software Design Analysis Report template (sections A-G). No framework choice, training outcome, deployment, or executed test is asserted.